Unit: 1

History of Java

- Developed by James Gosling in Sun Microsystems. Started working around 1990.
- First Version Released in 1995.
- Second version in 1998.
- Sixth version in 2006.
- Oracle takeover Sun Microsystem due to financial condition.
- 7th version Released by Oracle Inc USA.
- 8th version in 2014.
- 9th version in 2017. [only for 64-bit OS]
  [new versions in every 6th month in March, September]
- LTS version: long-term ~~state~~ support duration of 2-3 years.
- Latest version is 24 and last LTS version is 25.

Unit-1

Why Java?

→ Java is a programming language and a platform. It is a high-level, robust, object-oriented and secure programming language.

→ Java has a runtime environment and API, it is called a platform. Platform Independence, Object-Oriented, Community Support, Enterprise Adoption. JIT compilation.

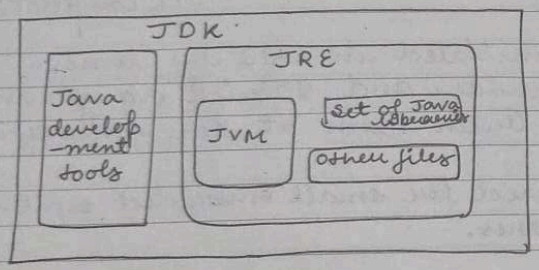
History of Java

- Java project initiated in 1991 by James Gosling, Mike Sheriden and Patrick Naughton known as the Green Team at Sun Microsystems.
- Initially designed for small embedded systems like set-up boxes.
- Originally called "Green talk" with file extension .gt.
- Renamed to Oak (Oak symbolizes strength)
- Renamed to Java in 1995 due to trademark issue with Oak technologies.
- Named after Java Island in Indonesia, where the first coffee was produced.
- Recognized by Time Magazine as one of the Ten best products of 1995.

Features of Java:

→ Simple
→ Object-Oriented : reusable codes through classes and objects
→ Robust
→ Multi-threaded : Supports concurrent execution
→ Platform Independent [WORA]
→ Secured : bytecode verification

JDK Vs JVM Vs JRE



• JDK CPUJ
• It's a software development kit required to developed Java applications.
• Include the JRE, an interpreter, a compiler and other tools needed for Java development.
• JDK is superset of JRE.

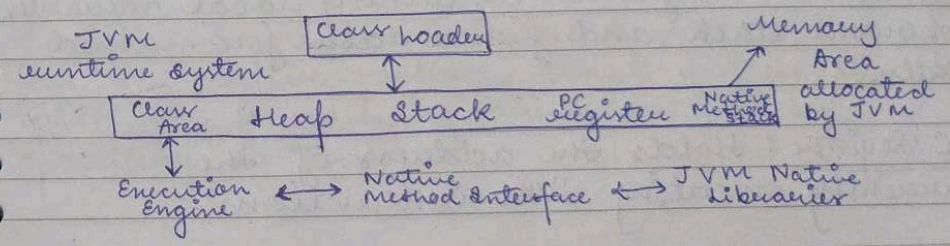
• JRE
• It's a part of the JDK but can be downloaded separately.
• Provides the libraries, the JVM and other components to run applications.
• Does not have tools and utilizes for developers like compilers.

• JVM
• It is an abstract machine. It is a specification that provides runtime environment in which Java bytecode can be executed. It is platform dependent.
JVM performs following operation:
→ Loads code
→ verifies code
→ Executes code
→ Provides runtime environment

JVM architecture

CEMN
CHS PN

Let's understand what basically Java architecture are designed to execute Java programs and provide platform independence.
It contains: class loader
Memory Area
Execution engine
Native Method Interface.



load → store

• Class Loader Subsystem

Purpose: loads class files into memory and performs memory management.
Components: has three main parts:
1. Bootstrap class loader: loads core java classes.
2. Extension class loader: loads classes from the Java Extensions directory.
3. Application class loader: loads classes from the classpath.

• Runtime Data Areas

Methods areas: Stores class structures like metadata, constant pool, method data, and field data.

Heap: Stores objects and arrays.

Java Stack: Stores frames containing local variables, operand stack, and frame data for method calls.

PC Register: Holds the address of the currently executing JVM instructions.

Native Method Stack: Manages native method calls used by Java programs.

• Execution Engine:

Interpreter: Executes bytecode instructions one at a time.

Just in Time Compiler: Compiles bytecode into native machine code for improved performance.

Garbage Collector: Manages memory by reclaiming the memory used by objects that are no longer reachable.

• Native Method Interface:

Purpose: Enables java code to interact with native applications and libraries written in C or C++

• Native Method Libraries:

Purpose: provides libraries required for the native methods that interact with the underlying hardware and operating system.

Java Source File Structure

A Java source file structure follows specific conventions and typically includes the following components:

Package declaration

- Specifies the package name
- must be the first element in the file if present.

```
package.com.example.myapp;
```

Import statements

- import other Java classes or package.
- must follow the package declaration and precede the class definition.

```
import java.util.List;
import java.util.ArrayList;
```

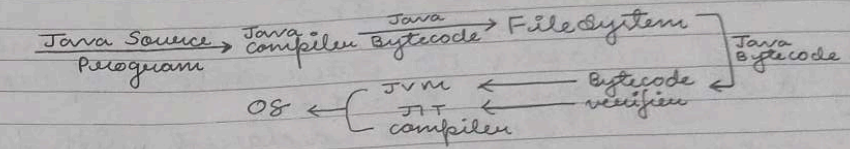
Class Interface declaration

- The main content of the source file, usually one public class/interface per file.
- The class name must match the file name.

```
public class MyClass {
        # class content
}
```

Compilation Process in Java

The compilation process in Java involves converting human-readable Java source code into bytecode that the JVM can execute.



Fundamentals

"Fundamentals" refer to the essential concepts and building blocks that form the core of the language.

1. Basic Syntax & Structure:

Keywords: Reserved words in Java that have a specific meaning in the language syntax (e.g. class, public, void)

Identifiers: Names used for classes, methods, variables, etc. Must begin with a letter, $, or an underscore followed by any combination of characters

Comments: Used to explain code, ignored by the compiler.

1. Single-line: // This is a comment.
2. Multi-line: /* This is a multi-line comment */

2. Data types
The various types of data that can be used in Java, such as primitive types (int, double, boolean, etc.) and reference types (objects and arrays).

3. Variables and constants
Declarations, initialization and usage of variables to store data temporarily during program execution.
Constants, declared with the final keyword, holds values that do not change.

4. Operators

5. Control structure : dictate the flow of a program.

6. Methods
Functions that encapsulate reusable codes and perform specific tasks. Methods are defined with a return type, name, parameters and a body that contains executable statements.

7. Classes and Objects
Classes serve as blueprints or templates for creating objects which are instances of classes.
Classes encapsulate data (fields) and behaviour (methods) related to a concept or entity.

8. Inheritance
The mechanism by which one class (subclass or derived class) inherits the properties and behaviours from another class (superclass or base class). Inheritance promotes code reuse and supports the "is-a" relationship.

9. Polymorphism
The ability of objects to take on multiple forms. In Java, polymorphism is achieved through method overriding (runtime polymorphism) and method overloading (compile-time polymorphism)

10. Abstraction
The concept of hiding complex implementation details and showing only the essential features of an object. It is only achieved through abstract classes and interfaces.

Programming Structures in Java

Defining classes in Java

Class: A blueprint or template that defines the properties (fields) and behaviours (methods) common to all objects and their types.

```
public class MyClass {
        // Fields
        // Constructor
        // Methods
}
```

Constructors

A constructor is a special block of code that runs when an object is created. It initializes the objects state.

→ Rules for creating Java constructor ←

- Constructor name must be the same as its class name.
- A constructor must have no explicit return type.
- A Java constructor cannot be abstract, static, final, and synchronized.

→ Types ←

1. Default Constructor:
Automatically provided by Java if no constructors are defined explicitly.

```
public class MyClass {
        public MyClass() {
                // Initialization code
        }
}
```

2. Parameterized Constructor:
Accepts parameters to initialize object fields with specific values. / that takes values as input.

```
class Student {
String name;
int age;
Student (
String n, int a) {
name = n;
age = a;
}
}
```

```
public class Car {
        private String model;
        private int year;
        // Parameterized constructor
        public Car (String model, int year) {
                this.model = model;
                this.year = year;
        }}.
```

Constructors are fundamentals in Java programming for creating and initializing objects, ensuring they are ready for use with the appropriate state.

3. No-argument Constructor

Constructors with no parameters (written by programmer)

```
class Car {
    Car() {
        SOPln("Car Created");
    }}
```

4. ~~Not~~ Copy constructor (Not built-in but created manually)

Java does not have built in constructor but we can create one.

```
class Student {
    String name.
    Student(Student s) {
        name = s.name;
    }
}
```

Methods in Java

It is a reusable block of code designed to perform specific tasks. By calling or invoking it, we can execute its functionality without needing to rewrite the code. This promotes code reusability, easy modification, and enhances code readability.

→ Method Components ←

- **Access Modifier**: defines the visibility of the method (public, private, protected).
- **Return Type**: the type of data the method returns (e.g string, void).
- **Method Name**: the name used to call the method.
- **Parameters**: Variables used to pass into the method.
- **Method Body**: the code that is to be executed when a method is called.

```
public class MyClass{
    //Method with no return values
    public void displayMessage(){
        SOPln("Hello World!");
    //Method with a return value
    public int addNumbers(int a, int b){
        return a+b;
    }}
```

Access Specifier

Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides four types of access specifier:
They are used for data hiding and encapsulation.

- Public: This method is accessible by all classes when we use public specifier in our application.
- Private: When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- Protected: When we use protected access specifier, the method is accessible within the same package or subclasses in a different package only through inheritance.
- Default: When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

Static Members

Static members belong to the class rather than to any specific instance of the class. They are shared among all instances of the class.
Static members can be accessed directly using the class name without needing to instantiate an object.

```
class Student {
    static String college = "RR Group";
    int rollNo;
    Student (int r) {
        rollNo = r;
    }
    void display() {
        SOPln (rollNo + " " + college);
    }
}
```

Final Members

They are constants that cannot be changed once initialized.
For variables, once a final variable is declared it cannot be reassigned.
For methods, a final method cannot be overridden by subclasses.
For classes, a final class cannot be subclassed.
Final members are often used to define constants or to prevent further modification of certain elements in a program.

Ex: String class in Java is final.

```
class Demo {
    final int x = 10;
    void change() {
        //x = 20;
    }
}
```

```
class Parent {
    final void show() {
        SOPln("Final Method");
    }
}
class Child Extends Parents {
}
```

Data types

Data types specify the different sizes & values that can be stored in the variable. They are of basically two types:

Primitive data types:
Boolean, char, byte, short, int, long, float and double.

Non-primitive data types: Classes, interfaces and arrays

Variables

It is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.

Operators

Arithmetic Operators: +, -, *, /, %
Relational Operators: ==, !=, <, >, <=, >=
Logical Operators: &&, ||, !
Assignment Operators: =, +=, -=, *=, /=
Increment / Decrement Operators: ++, --

Control Flow:

Java compiler executes the code from top to bottom.
Java provides statement that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java